★ 1. Introduction To REST API ★

1. What is an API ?

API (Application Programming Interface) is a set of rules and protocols that allows two applications to communicate with each other. It acts as an intermediary between client and server.

2. What is REST ?

REST (Representational State Transfer) is an architectural style for designing networked applications. It uses standard HTTP methods to perform CRUD operations on resources.

3. REST vs SOAP

REST	SOAP
Architectural Style	Protocol
Uses HTTP	Uses HTTP, SMTP, etc.
Lightweight	Heavyweight
Supports multiple formats (JSON, XML, etc.)	Supports only XML
Stateless	Can be Stateful
Better performance and scalability	Low performance compared to REST
Widely used in Web and Mobile Apps	Used in Enterprise applications

4. Client-Server Architecture

In REST, client and server are independent. Client (UI / App) sends request to server, server processes it and sends response.

- Client : Sends request, handles UI.
- Server : Processes request, manages data, sends response.

5. Resource-based Architecture

Everything in REST is a resource.
Each resource is identified by a unique URI.
Example :

Resources	Representations
• /users	• JSON
• /users/1	• XML
• /products/10	• Plain Text
• /orders/100	• HTML

URI identifies the resource and representation represents the current state of that resource.

6. REST Constraints

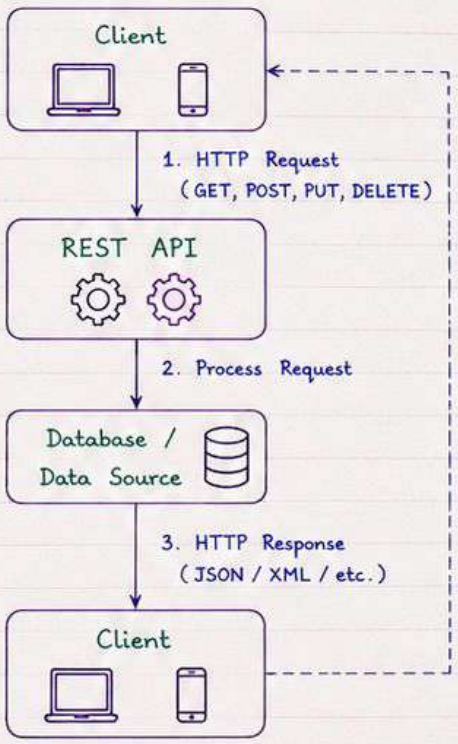
REST is based on the following constraints :

1. Client-Server : Separation of concerns between client and server.
2. Stateless : Each request from client must contain all the information required. Server does not store any client state.
3. Cacheable : Responses must define themselves as cacheable or non-cacheable.
4. Uniform Interface : A consistent way for client to interact with server. (Resources, Methods, Representations)
5. Layered System : Client cannot tell whether it is connected directly to the end server or to an intermediary.

7. Why REST is Widely Used ?

- Simple and easy to understand
- Uses standard HTTP methods
- Lightweight and flexible
- Supports multiple data formats (JSON, XML)
- Stateless → better scalability
- Works well with mobile and web applications
- Easy to test and maintain

8. How REST API Works ?



★ 2. HTTP METHODS ★

HTTP methods (or verbs) are used to perform operations on resources identified by a URI. Each method has a specific purpose.

Method	Purpose	Idempotent (Same result if called multiple times)	Safe (No server side changes)	Example	Typical Use
GET (Read)	Retrieve data from the server.	Yes	Yes	GET /users GET /users/10	• Get all users • Get user by id
POST (Create)	Create a new resource.	No	No	POST /users	• Create a new user • Submit form data
PUT (Update)	Update or replace an existing resource entirely.	Yes	No	PUT /users/10	• Update user (full) • Replace user info
PATCH (Partial Update)	Partially update an existing resource.	Yes	No	PATCH /users/10	• Update user (partial) • Change single field
DELETE (Delete)	Delete a resource from the server.	Yes	No	DELETE /users/10	• Delete user • Remove resource
OPTIONS (Info)	Get information about supported methods.	Yes	Yes	OPTIONS /users	• Get allowed methods • CORS preflight
HEAD (Rertial Headers)	Similar to GET but returns headers only (no body).	Yes	Yes	HEAD /users/10	• Check resource existence • Get metadata

Idempotent Methods

- GET, PUT, DELETE, HEAD, OPTIONS are idempotent.
- Calling them multiple times produces the same result without additional side effects.

Example :
PUT /users/10 with same data 5 times results in the same user data.

Safe Methods
Methods that do not change the state on the server.
Safe Methods : GET, HEAD, OPTIONS

Quick Comparison

Method	Create	Read	Update	Delete	Idempotent	Safe
GET	–	✓	–	–	✓	✓
POST	✓	–	–	–	✗	✗
PUT	–	–	✓	–	✓	✗
PATCH	–	–	✓	–	✓	✗
DELETE	–	–	–	✓	✓	✗
OPTIONS	–	–	–	–	✓	✓
HEAD	–	✓	–	–	✓	✓

Interview Tip
- Use the correct HTTP method for the operation.
- GET should not change data.
- PUT replaces the entire resource, PATCH updates partially.

★ 3. HTTP STATUS CODES ★

HTTP status codes are returned by the server in response to a client's request. They indicate whether the request was successful or not.

1. 2xx – Success

200	OK Request was successful.
201	Created Resource created successfully.
204	No Content Request successful but no content to return.
205	Reset Content Request successful. Reset the form.

2. 3xx – Redirection

301	Moved Permanently Resource moved permanently to a new URL.
302	Found Resource found but temporarily moved to another URL.
304	Not Modified Resource not modified since the last request.
307	Temporary Redirect Similar to 302 but method and body are not changed.

3. 4xx – Client Errors

400	Bad Request Server cannot process the request due to invalid syntax.
401	Unauthorized Authentication is required and has failed or not been provided.
403	Forbidden Server understood the request but refuses to authorize it.
404	Not Found The requested resource could not be found.
405	Method Not Allowed The request method is known by the server but not supported by the resource.
409	Conflict Request could not be completed due to a conflict with the current state.
422	Unprocessable Entity Server understands the request but cannot process it due to semantic errors.

4. 5xx – Server Errors

500	Internal Server Error An unexpected condition was encountered on the server.
502	Bad Gateway Server, while acting as a gateway or proxy, received an invalid response.
503	Service Unavailable Server is not ready to handle the request. Temporarily overloaded or down.
504	Gateway Timeout Server, while acting as a gateway or proxy, did not receive a timely response.
505	HTTP Version Not Supported Server does not support the HTTP protocol version used in the request.

Interview Tip

Do not return 200 OK for every request. Always return the most appropriate status code.

Summary

- 2xx → Success
- 3xx → Redirection
- 4xx → Client Error
- 5xx → Server Error

☆ Always use correct status codes. It improves API reliability and user experience. ☆

★ 4. REQUEST & RESPONSE ★

Every REST API communication is based on HTTP request from the client and HTTP response from the server.

1. HTTP REQUEST

A request contains all the information that the server needs to process the request.

① URL
Endpoint of the resource.
Example: POST /api/users

② Query Parameters (?)
Optional parameters to filter / sort / search.
Example: /api/users?page=1&limit=10

③ Path Parameters
Values included in the URL path.
Example: /api/users/10

④ Headers
Additional information about the request.
Example:
- Authorization: Bearer <token>
- Content-Type: application/json
- Accept: application/json

⑤ Body (Payload)
Data sent to the server (in POST, PUT, PATCH).
Usually in JSON format.

Example Request

```
POST /api/users HTTP/1.1
Host: localhost:5000
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
Content-Type: application/json
Accept: application/json
{
  "name": "John Doe",
  "email": "john@email.com",
  "age": 25,
  "isActive": true
}
```

→ Request Body (JSON)

2. HTTP RESPONSE

A response contains the result of the request along with status code and data.

① Status Line
HTTP version + status code + status message.

② Headers
Additional information about the response.

③ Body
The actual data returned by the server.
Usually in JSON format.

Example Response

```
HTTP/1.1 201 Created
Content-Type: application/json
Date: Sun, 12 May 2024 10:30:00 GMT
Content-Length: 112
{
  "id": 1,
  "name": "John Doe",
  "email": "john@email.com",
  "age": 25,
  "isActive": true,
  "createdAt": "2024-05-12T10:30:00Z"
}
```

→ Status Line
→ Headers
→ Body (JSON)

Common Response Examples

Status Code	Meaning	When It Occurs
200 OK	Success	Request successful (GET, PUT, PATCH)
201 Created	Resource created	New resource created (POST)
204 No Content	Success (no data)	Resource deleted (DELETE)
400 Bad Request	Invalid request	Invalid data / parameters
401 Unauthorized	Authentication required	No token / invalid token
403 Forbidden	Access denied	Not allowed to access resource
404 Not Found	Resource not found	Resource does not exist
500 Internal Server Error	Server error	Unexpected server error

Important Notes ☆

- ✓ Headers and body are optional in some methods (like GET).
- ✓ Always send data in JSON format.
- ✓ Use appropriate HTTP method for the operation.
- ✓ Keep requests stateless.

Flow of Communication

Interview Tip

Always validate the request, return correct status code and meaningful response.

☆ Good request and response design makes your API easy to use and maintain. ☆

★ 5. REST API Best Practices ★

1. Use Nouns, not Verbs

URIs should represent resources, not actions.

Use nouns to represent collections and specific items.

✕ Avoid (Verbs in URI)

```
/getUsers
/createUser
/deleteUser/10
```

✓ Use (Nouns in URI)

```
/users
/users
/users/10
```

2. Use Plural Nouns

Always use plural nouns for resources.

Examples :

- /users (collection)
- /users/10 (specific user)
- /products
- /orders
- /categories/5/products

3. Use Proper HTTP Methods

Use the correct HTTP methods for the operations.

Method	Purpose	Idempotent	Safe
GET	Retrieve data	Yes	Yes
POST	Create data	No	No
PUT	Update entire resource	Yes	No
PATCH	Update part of resource	No	No
DELETE	Delete resource	Yes	No

4. Filtering, Sorting & Searching

- Use query parameters for filtering, sorting and searching.
- Keep it simple and consistent.

Examples :

```
/users?role=admin
/users?status=active&role=admin
/users?sort=name&order=asc
/users?search=john
/products?category=books&sort=price&order=desc
```

5. Pagination

Use pagination to return large datasets.

Always provide total count (if possible).

Examples :

```
/users?page=1&limit=10
/users?page=2&limit=10&sort=name
/users?limit=20&offset=40
```

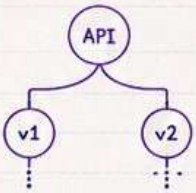
|< < 1 2 3 > >|

6. Version Your API

Versioning helps manage changes without breaking existing clients.

Examples :

```
/api/v1/users
/api/v2/users
/api/v1/products/10
```



7. Consistent Error Handling

- Return proper status codes.
- Provide meaningful error messages.
- Keep error response consistent.

Example Error Response (JSON)

```
{
  "success": false,
  "statusCode": 404,
  "message": "User not found",
  "errors": []
}
```

8. Use Standard Response Format

Keep successful responses consistent across the API.

Return relevant data only.

Example Success Response (JSON)

```
{
  "success": true,
  "message": "User retrieved successfully",
  "data": {
    "id": 10,
    "name": "John Doe",
    "email": "john@email.com"
  }
}
```

9. Follow Security Best Practices

Use HTTPS Always.

Authenticate requests.

Authorize access.

Validate all inputs.

Prevent common attacks (SQLi, XSS, CSRF).

Rate limiting to prevent abuse.

Never expose sensitive data.

☆ Follow best practices to build clean, secure, scalable and maintainable APIs. ☆

★ 6. AUTHENTICATION & SECURITY ★

Security is crucial in REST APIs to protect data, prevent unauthorized access and ensure integrity and privacy.

1. Authentication vs Authorization

- **Authentication** : Verifies who the user is. (Are you who you claim to be?)
- **Authorization** : Verifies what the user is allowed to do. (Do you have permission?)

Authentication	Authorization
Confirms identity (login)	Checks permissions (access control)
Examples : Login, JWT, API Key	Examples : Roles, Policies, Claims

2. Authentication Methods

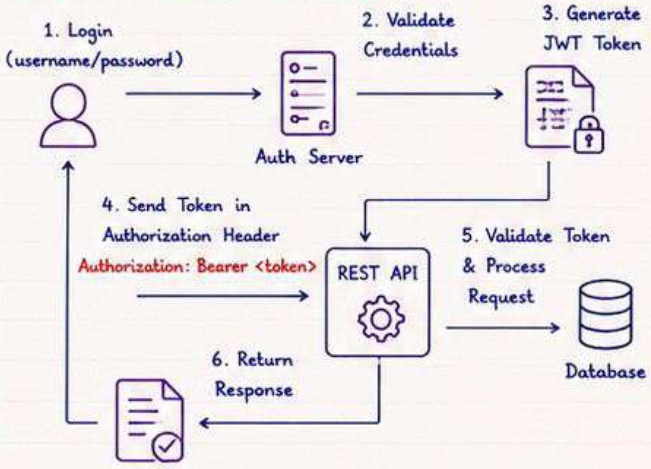
① **JWT (JSON Web Token)**
- Stateless authentication mechanism.
- Token contains user claims and is digitally signed.

② **Bearer Token**
- Client sends token in Authorization header.
- Format : `Authorization: Bearer <token>`

③ **OAuth 2.0**
- Authorization framework for delegated access.
- Common in third-party login (Google, GitHub).

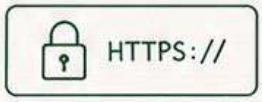
④ **API Key**
- Simple key passed in header or query.
- Use for server-to-server communication.

3. JWT Authentication Flow



☆ JWT is stateless → No need to store sessions on the server.

4. HTTPS (SSL/TLS)



- Always use HTTPS.
- Encrypts data in transit.
- Prevents eavesdropping and man-in-the-middle attacks.

5. CORS (Cross-Origin Resource Sharing)

Allows or restricts resources to be requested from a domain different than the API domain.

```
Access-Control-Allow-Origin : https://example.com
Access-Control-Allow-Methods : GET, POST, PUT, DELETE
Access-Control-Allow-Headers : Content-Type, Authorization
```

6. Rate Limiting

- Limit the number of requests a client can make in a specific time.
- Prevents abuse and brute force attacks.
- Implement using IP, user or API key.

7. Input Validation

- Validate all inputs on server side.
- Prevents injection attacks.
- Check data type, length, format, range, etc.

8. Common Security Threats

• SQL Injection	→ Use parameterized queries
• XSS (Cross Site Scripting)	→ Encode output
• CSRF (Cross Site Request Forgery)	→ Use anti-forgery tokens
• Broken Authentication	→ Use strong auth (JWT/OAuth)
• Sensitive Data Exposure	→ Encrypt sensitive data
• Mass Assignment	→ Validate and whitelist fields
• Security Misconfiguration	→ Secure configurations

9. Other Best Practices

- ✓ Use strong passwords and hashing (BCrypt, Argon2).
- ✓ Use short-lived tokens and refresh tokens.
- ✓ Store secrets in environment variables.
- ✓ Log security events and monitor.
- ✓ Keep dependencies and libraries updated.
- ✓ Implement proper error handling (no sensitive info).

☆ Security is not optional in APIs. Build secure APIs to protect your users and data. ☆

★ REST API Best Practices ★

1. Use proper resource naming

- Use nouns and plural form for resources.
- Use nested resources for relationships.

Examples :

- ✓ GET /users
- ✓ GET /users/123
- ✓ GET /users/123/orders
- ✓ GET /products
- ✓ GET /products/456/reviews

2. Use meaningful HTTP status codes

- Use correct status codes to represent the result.
- Helps clients handle responses accurately.

Common Status Codes :

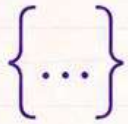
Code	Meaning
200	OK (Success)
201	Created (Resource created)
204	No Content (Success, no body)
400	Bad Request (Invalid input)
401	Unauthorized (Auth required)
403	Forbidden (Access denied)
404	Not Found (Resource not found)
500	Internal Server Error (Server issue)

3. Use consistent response format

- Always return responses in a consistent structure.
- Makes it easier for clients to parse and handle.

Example (JSON) :

```
{
  "success": true,
  "data": {
    "id": 123,
    "name": "John Doe",
    "email": "john@example.com"
  },
  "message": "User retrieved successfully"
}
```

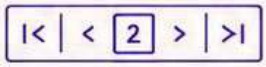


4. Implement pagination

- Return large data in small chunks.
- Improve performance and user experience.

Example : GET /users?page=2&limit=10

```
{
  "data": [ ... ],
  "pagination": {
    "page": 2,
    "limit": 10,
    "total": 150,
    "totalPages": 15
  }
}
```



5. Filter, sort & search

- Allow clients to filter, sort and search data.
- Use query parameters.

Example :

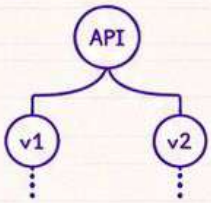
```
GET /users?role=admin&status=active
GET /users?sort=name&order=asc
GET /products?search=phone
GET /orders?dateFrom=2024-01-01&dateTo=2024-01-31
```

6. Version your API

- Handle breaking changes without affecting clients.
- Include version in the URI.

Examples :

```
/api/v1/users
/api/v1/products
/api/v2/users
```



7. Authenticate & authorize requests

- Protect your APIs from unauthorized access.
- Validate user and permissions.

Best Practices :

- ✓ Use OAuth 2.0 / JWT for authentication.
- ✓ Use roles/permissions for authorization.
- ✓ Never trust client-side data.

8. Validate all inputs

- Validate and sanitize all inputs.
- Prevent injection attacks and bad data.

Checklist :

- ✓ Validate data type, length, range.
- ✓ Check required fields.
- ✓ Sanitize strings.
- ✓ Return proper error if invalid.

9. Rate limiting & throttling

- Prevent abuse and ensure fair usage.
- Limit the number of requests.

Example :

Limit: 100 requests / IP / 15 minutes

10. Log & monitor your APIs

- Logging helps in debugging and auditing.
- Monitor performance and errors.

Log Important Things :

- ✓ Request method & URL
- ✓ Request & response time
- ✓ Status code
- ✓ Errors & exceptions
- ✓ User / IP

☆  Best APIs are simple, consistent, secure and well documented. Follow these best practices to build reliable and scalable APIs. ☆

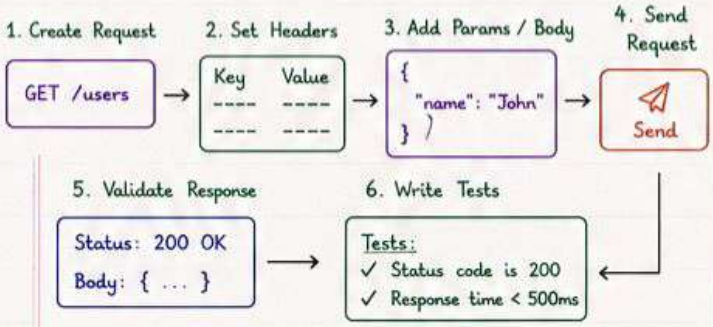
Testing and documenting your API ensures reliability, usability and easy integration for developers.

1. API Testing Tools

Popular tools to test REST APIs.

- Postman – Send requests, view responses, set headers, tests.
- Swagger UI – Interactive API testing.
- curl – Command line tool.
- Insomnia – Lightweight REST client.
- Thunder Client – VS Code extension.

2. Testing in Postman (Workflow)



3. Sample Postman Request

```
GET    https://api.example.com/users/1
Headers:
Authorization: Bearer <token>
Accept: application/json

Query Params:
?include=orders
```

POSTMAN

Sample Response

```
HTTP/1.1 200 OK
Content-Type: application/json
{
  "id": 1,
  "name": "John Doe",
  "email": "john@email.com"
}
```

Validate status & data

4. Automated Testing (Postman Tests)

Write test scripts to validate the response.

```
pm.test("Status code is 200", function () {
  pm.response.to.have.status(200);
});

pm.test("Response time is less than 500ms", function (){
 pm.expect(pm.response.responseTime).to.be.below(500);
});

pm.test("Response has name", function () {
  var jsonData = pm.response.json();
  pm.expect(jsonData.name).to.exist;
});
```

Automate tests to ensure consistency and reliability.

5. API Documentation

Good documentation helps developers understand how to use your API.

What to Document?

- ✓ Base URL
- ✓ Authentication
- ✓ Endpoints
- ✓ Request (Headers, Params, Body)
- ✓ Response (Status codes, Body)
- ✓ Error Codes
- ✓ Examples

6. OpenAPI / Swagger

- OpenAPI (formerly Swagger) is a specification for documenting REST APIs.
- It allows automatic generation of interactive API docs (Swagger UI).
- Supports request/response schema, models, authentication and much more.

Example (OpenAPI - YAML)

```
openapi: 3.0.0
info:
  title: User API
  version: 1.0.0
paths:
  /users:
    get:
      summary: Get all users
      responses:
        '200':
          description: Successful response
          content:
            application/json:
              schema:
                type: array
                items:
                  $ref: '#/components/schemas/User'
components:
  schemas:
    User:
      type: object
      properties:
        id:
          type: integer
        name:
          type: string
        email:
          type: string
```

Defines endpoints, schemas, responses and more.

Best Practices

- ✓ Test all endpoints (positive & negative cases).
- ✓ Validate status codes and response data.
- ✓ Use environment variables in Postman.
- ✓ Automate tests for regression testing.
- ✓ Keep documentation updated with API changes.